

```

0000 SEPARATE (IDL) ::= --
E DESCRIPTION IDL EN MEMOIRE VIRTUELLE FILENAME: FILE_TYPE; FICHER IDL SLINE:= STRING ( 1..256 ) LIGNE-TEXTE COURANTE COL:=
NATURAL:= 1; PROCHAINE COLONNE LIRE F_COL:= NATURAL; PREMIERE COLONNE DU LEXEME TOKEN_LENGTH:= NATURAL; LONGUEUR
UR DU LEXEME LAST:= NATURAL:= 0; NOMBRE DE CARACTERE DE LA LIGNE TOKEN_IS_NAME:= BOOLEAN; LINE_COUNT:= NATURAL:= 0; ATTR_COUN
T:= INTEGER:= -1; SOURCE_LIST:= SEQ_TYPE; LISTE DES LIGNES SOURCES SOURCEPOS:= TREE; LA POSITION SOURCE DU LEXEME COURANT
SOURCELINE:= TREE; LE NOUVEAU LIGNE SOURCE USER_ROOT:= TREE; RACINE DE L'ARBRE TYPE CONTEXT_TYPE:= IS (NIL, IN_NODE, IN_CL
ASS); CONTEXT:= CONTEXT_TYPE:= NIL; PROCEDURE PROCESS_IDL IS
PROCEDURE GET_TOKEN IS BEGIN WHILE COL <= LAST A
ND THEN ( SLINE( COL ) = ' ' OR ELSE-SLINE( COL ) = ASCII.HT ) LOOP PASSER LES ESPACES COL := COL + 1; END LOOP; IF COL < LAST AN
D THEN SLINE( COL ) = '-' AND-THEN SLINE( COL+1 ) = '-' THEN SAUTER EN FIN DE LIGNE SUR COMMENTAIRE COL := LAST +1; END IF; IF CO
L > LAST THEN RETURN; SI ON-EST FIN DE LIGNE LOOP IF END_OF_FILE ( IFILE ) THEN RETURN; END IF; IF END_OF_LINE ( IFILE
) THEN SKIP-LINE( IFILE ); LINE_COUNT := LINE_COUNT + 1; LAST := 0; ELSE-SLINE( 1..2 ) = "???" FORCER AUTRE CHOSE // EN C
AS DE LIGNE VIDEO GET-LINE( IFILE, SLINE, LAST ); LIRE UNE LIGNE-IF SLINE( 1..2 ) = "/" THEN NE S'OCCUPER QUE DES LIGNES COMMENCA
T PAR / COL := 3; SE METTRE EN COL 3 POST "/"-ELSE- COL := LAST + 1; POST FIN DE LIGNE POUR NEGLIGENCE LA LIGNE-END IF;
END IF; WHILE COL <= LAST AND-THEN ( SLINE( COL ) = ' ' OR ELSE-SLINE( COL ) = ASCII.HT ) LOOP PASSER LES BLANCS COL := COL + 1;
END LOOP; IF COL <= LAST THEN IF SLINE( COL ) = '-' AND THEN COL < LAST AND THEN SLINE( COL+1 ) = '-' THEN COL := LAST + 1; ELSE-
LIGNE-NON VIDEO SOURCELINE:= MAKE ( DN_SOURCELINE ); CREER-UN NOUVEAU LIGNE SOURCE DI ( XD_USER_ROOT, SOURCELINE, LINE_COUNT );
Y METTRE LE NUMERO DE LIGNE LIST ( SOURCELINE, (TREE_NIL, TREE_NIL); INITIALISER LA XD_ERROR_LIST SOURCE_LIST := APPEND ( SOURCE_L
IST, SOURCELINE ); METTRE LA LIGNE-EN FILE EXIT; END IF; END IF; END LOOP; F_COL:= COL; TOKEN_LENGTH :=
1; TOKEN_IS_NAME := FALSE; CASE SLINE( COL ) IS WHEN 'A'..'Z' 'a'..'z' '>' THEN TOKEN_IS_NAME:= TRUE; COL:= COL + 1; WHILE
COL <= LAST AND THEN ( SLINE( COL ) IN 'A'..'Z' 'a'..'z' '0'..'9' OR ELSE-SLINE( COL ) IN 'A'..'Z' 'a'..'z' '0'..'9' OR ELSE SLINE( COL ) IN
'0'..'9' ) LOOP TOKEN_LENGTH := TOKEN_LENGTH + 1; COL := COL + 1; END LOOP; WHEN '!' >> PEUT ETRE :=: IF COL + 2 <= LAST AND THEN SLINE( COL+1 ) = '!' AND THEN SLINE( COL+2 ) = '!' THEN OUI UN TOKEN_LENGTH := 3; LONGUEUR 3; COL := COL + 3; ELSE COL := COL + 1; END IF; WHEN '=' >> PEUT ETRE :=: IF COL + 1 <
= LAST AND THEN SLINE( COL+1 ) = '=' THEN OUI UN TOKEN_LENGTH := 1; ELSE COL := COL + 1; END IF; WHEN '>' >> COL:= COL + 1; WHEN OTHERS >> COL:= COL + 1; END CASE; SOURCEPOS := MAKE_SOURCE_PO
SITION( SOURCELINE, SRCOL_IDX( F_COL ); END GET_TOKEN;
PROCEDURE PROCESS_IDL IS RULE_NODE:= TREE; NODE_LIST:= SEQ_TYPE:= (TREE_NIL, TREE_NIL); PRIOR
R_F_COL:= POSITIVE:= 1; PRIOR_TOKEN_LENGTH:= NATURAL:= 0; TYPE ATTR_TYPE:= IS (NORMAL, SEQ);
PROCEDURE MAKE_RULE_OR_CLASS_NODE IS PROCEDURE MAKE_RULE_OR_CLASS_NODE ( NODE_NAME:=
STRING ) IS SYMBOL:= STORE_SYM ( NODE_NAME ); STOCKER/RETIRER LE NOM DE-NOUVEAU REGLE (NOM EN-PARTIE GAUCHE AVANT-LE >) R_
LIST:= SEQ_TYPE:= LIST (SYMBOL); LISTE-CONTENANT-LE NOUVEAU REGLE ASSOCIEES AU SYMBOLE (REFERENCE COMME GAUCHE-DE REGLE OU TYPAGE) BEGINS
IF IS_EMPTY (R_LIST) THEN LA LISTE EST VIDE, C'EST LA PREMIERE DEFINITION DE REGLE/CLASSE RULE_NODE := MAKE ( DN_CLASS_NODE
); FABRIQUER UN NOUVEAU NOEUDE POUR LA REGLE/CLASSE D( XD_SYMPREP, RULE_NODE, SYMBOL ); METTRE LE SYMBOLE DU NOM DE REGLE (NOM GAUCHE
) DANS LE XD_NAME DU-NOUVEAU REGLE LIST ( RULE_NODE, (TREE_NIL, TREE_NIL); INITIALISER VIDE LA LISTE DES ELEMENTS DU CTE DROIT
IF CONTEXT = IN_NODE THEN DANS UN NOUVEAU REGLE SIMPLE DB ( XD_IS_CLASS, RULE_NODE, FALSE ); MARQUER UNE REGLE DE STRUCTURE D'ATTR
IBUS ELSE- DANS UN CLASSE DB ( XD_IS_CLASS, RULE_NODE, TRUE ); MARQUER UNE REGLE DE DEFINITION DE CLASSE END IF
D( LX_SRCPOS, RULE_NODE, SOURCEPOS ); METTRE LA POSITION SOURCE-DONNEE PAR GET_TOKEN D( XD_PARENT, RULE_NODE, TREE_VOID
); LIST (SYMBOL, APPEND( (TREE_NIL, TREE_NIL), RULE_NODE ); METTRE LE NOUVEAU-REGLE COMME ELEMENT-UNIQUE DE-LISTE DU SYMBOLE DE-NOM GAU
CHE (XD_DEFLIST) NODE_LIST := APPEND (NODE_LIST, RULE_NODE); METTRE EN LISTE-LA REGLE ELSE- UNE REGLE AVEC MEMBRE PA
RTIE GAUCHE A-DEJ ETE VUE DECLARE-DEFINIR: TREE:= HEAD (R_LIST); PRENDRE LE NOUVEAU DE TETE, LE NOUVEAU REGLE BEGINS-IF DEF.TY:=
DN_CLASS_NODE THEN VERIFIER CE C'EST UNE REGLE- RULE_NODE := DEF; PRENDRE CE NOUVEAU COMME COURANT-IF CONTEXT = IN_CLASS THEN
ON A UN NOM CITE EN CLASSE (PEUT AVOIR ETE TROUVE AVANT-UN CITE-COMME TYPAGE EN DROITE DE REGLE D'ATTRIBUTION) DB (XD_IS_CLASS, RU
LE_NODE, TRUE ); MARQUER COMME CLASSE- END IF; ELSE- ANOMALIE LE NOUVEAU N'EST PAS UNE REGLE- ERROR (-SOURCEPOS, "HOLA ! " & NO
DE_NAME & " N EST PAS UN NOM EN PARTIE GAUCHE (REGLE) !" ); END IF; END IF; END MAKE_RULE_OR_CLASS_NODE;
PROCEDURE MAKE_ATTR PROCEDURE MAKE_ATTR ( ATTR_NAME:= STR
ING; ATYPE:= ATTR_TYPE; NOM_TYPE:= STRING ) IS SYMBOL:= STORE_SYM ( ATTR_NAME ); STOCKER/RETIRER LE SYMBOLE (SYMREP)-DE L'ATTRIBUT
A LIST:= SEQ_TYPE:= LIST (SYMBOL); LISTE-CONTENANT-LA DEFINITION DE L'ATTRIBUT TYPAGE:= TREE:= STORE_SYM (NOM_TYPE );
STOCKER/PRENDRE LE SYMBOLE DU TYPE DE L'ATTRIBUT (UN NOM DE REGLE OU DE CLASSE) ATTR:= TREE; IF IS_EMPTY (A_LIST) THEN
AUCUNE APPARITION DE CET ATTRIBUT ATTR_COUNT := ATTR_COUNT + 1; UN ATTRIBUT DE PLUS ATTR := MAKE ( DN_ATTR );
CREER-UN NOUVEAU DE TYPE ATTRIBUT/TERMINAL D( XD_SYMPREP, ATTR, SYMBOL ); POINTER LE SYMBOLE DANS LE CHAMP XD_SYMPREP DU-TERMINAL
CASE ATYPE IS SUIVANT QUE L'ON A UN ATTRIBUT SIMPLE OU UNE SEQUENCE WHEN NORMAL= ATTRIBUT SIMPLE-DI ( XD
ATTR_ID, ATTR, ATTR_COUNT ); PORTER LE N D'ATTRIBUT EN POSITIF WHEN OTHERS= ATTRIBUT SEQUENCE, PORTER LE N D'ATTRIBUT
EN NEGATIF-DI (XD_ATTR_ID, ATTR, -ATTR_COUNT ); END CASE; D( XD_ATTR_TYPER, ATTR, TYPAGE ); METTRE LE TYPAGE LIST
(SYMBOL, APPEND( (TREE_NIL, TREE_NIL), ATTR ); PORTER L'ATTRIBUT CRE EN-LISTE DANS LE XD_DEFLIST DU SYMPREP ELSE- IL Y A DE
J EU UNE APPARITION ATTR := HEAD (A_LIST); PRENDRE LA DEFINITION EXISTANTE END IF; IF ATTR.TY := DN_ATTR THEN
CE DOIT ETRE UN-ATTRIBUT/TERMINAL CASE ATYPE IS WHEN NORMAL= ATTRIBUT SIMPLE-(NON SEQUENCE)-IF DI ( XD_ATTR_ID, ATTR )
< 0 THEN IL Y A UNE ANOMALIE (LE N D'ATTRIBUT DOIT ETRE POSITIF-DANS CE CAS) ERROR (-SOURCEPOS, "ATTR IS SEQ & ATTR NAME"); ELSE-IF TYPA
GE /= D( XD_ATTR_TYPER, ATTR ) THEN ANOMALIE SI LE TYPAGE NE CORRESPOND-PAS CE QUI EST DANS L'ATTRIBUT (VOIR LA LIGNE (**) UN PEU-PLUS HAUT)
DECLARE OLD_TYPAGE:= TREE:= D( XD_ATTR_TYPER, ATTR ); BEGINS-IF ERROR(-SOURCEPOS, "VALUE OF " & ATTR_NAME & " IS " & PRINT_NAME ( T
YPAGE ) & " != PAGE_IDX IMAGE (TYPAGE.PG) & " & LINE_IDX IMAGE (TYPAGE.LN) & " & NODE_NAME IMAGE (TYPAGE.TY) & " CALLED " & PRIN
T_NAME (TYPAGE) & " NOT " & PRINT_NAME (OLD_TYPAGE) & " & PAGE_IDX IMAGE (OLD_TYPAGE.PG) & " & LINE_IDX IMAGE (OLD_TYPAGE.LN) & " &
& " & NODE_NAME IMAGE (OLD_TYPAGE.TY) & " CALLED " & PRINT_NAME (OLD_TYPAGE) & " ) WHEN OTHERS= ATTRIB
UT SEQUENCE-IF DI (XD_ATTR_ID, ATTR) >= 0 THEN ANOMALIE SI LE N D'ATTRIBUT EST POSITIF (IL DOIT ETRE NEGATIF POUR UNE SEQUENCE) ERROR (-
SOURCEPOS, "ATTR IS NOT SEQ & ATTR NAME"); ELSE-IF TYPAGE /= D( XD_ATTR_TYPER, ATTR ) THEN SI LE-TYPAGE DE-SEQUENCE DIFFERE DE-L'ANTERIEUR-
DECLARE TEMP:= TREE:= ATTR; GARER-L'ATTRIBUT ANCIENNEMENT TYPE- BEGINS ATTR := MAKE ( DN_ATTR ); REFAIRE UN TERMINAL Q
UI EST PRIS COMME ATTRIBUT D( XD_SYMPREP, ATTR, D( XD_SYMPREP, TEMP ); DANS LE SYMPREP DU NOUVEAU, RECOPIER LE SYMPREP-DE L'ANCIEN D
I (XD_ATTR_ID, ATTR, DI( XD_ATTR_ID, TEMP ); REPORTER AUSSI LE N D'ATTRIBUT D( XD_ATTR_TYPER, ATTR, TYPAGE ); METTRE LE TYPA
GE DANS CE NOUVEAU NOUVEAU ATTRIBUT QUI DIFFERE PAR LE-TYPAGE END; END IF; DECLARE-ASEQ:= SEQ_TYPE:= LIST (RULE_NODE)
PRENDRE LA XD_LIST DU NOUVEAU REGLE-EN COURS BEGINS-LIST (RULE_NODE, APPEND (ASEQ, ATTR ); AJOUTER AU NOUVEAU REGLE EN-COURS
LA LISTE DES ATTRIBUTS AUGMENTEE END; ELSE- LA DEFINITION (TROUVEE) N'EST PAS UN TERMINAL-! ERROR (SOURCEPOS, "NOT
DEFINED AS AN ATTRIBUTE " & ATTR_NAME ); END IF; END MAKE_ATTR;
PROCEDURE MAKE_MEMBER PROCEDURE MAKE_MEMBER ( MEMBER_NAME:= STRING ) IS SYMBOL:= STORE_SYM (
MEMBER_NAME ); STOCKER/PRENDRE LE SYMPREP CORRESPONDANT AU NOM D'ELEMENT DE CLASSE MEMBER:= TREE:= MAKE (DN_MEMBER); FABRIQUER
UN NON TERMINAL-POUR UN ELEMENT APPARTENANT -UNE CLASSE M_LIST:= SEQ_TYPE:= LIST (SYMBOL); LA LISTE CONTENANT LA REGLE DEFINISSANT L
E SYMBOLE BEGINS D( XD_SYMPREP, MEMBER, SYMBOL ); METTRE LE SYMPREP DANS LE CHAMP XD_SYMPREP DU MEMBRE DE CLASSE D( LX_SRCPOS,
MEMBER, SOURCEPOS ); METTRE LA POSITION SOURCE-DU MEMBRE-CRE PAR GET_TOKEN LIST (RULE_NODE, APPEND (LIST (RULE_NODE), MEMBER));
AJOUTER LE MEMBRE LA LISTE DES MEMBRES DU NOUVEAU END IF;
BEGINS LAST := 0; COL := 1; GET_TOKEN; LOOP EXIT WHEN END_OF_FILE (IFILE) OR ELSE SLINE( F_COL.F_COL + TOKE
N_LENGTH - 1 ) = "end"; FINIR-AVEC LE FICHIER OU LE LEXEME %%% QUI INDIQUE LA FIN IF TOKEN_IS_NAME THEN LEXEME IDENTIFICATEUR
PRIOR_F_COL:= F_COL; GARDER SA POSITION PRIOR_TOKEN_LENGTH := -TOKEN_LENGTH; ET SA LONGUEUR GET_TOKEN;
ET PASSER AU SUIVANT (QUI-VA PERMETTRE DE SAVOIR CE QUE-L'ON VA FAIRE) ELSE-IF SLINE( F_COL.F_COL + TOKEN_LENGTH - 1 ) = "=" THEN I
NDIQUE UNE REGLE DEFINISSANT DES ATTRIBUTS CONTEXT := IN_NODE; GARDER UNE TRACE DE CE FAIT : DEFINITION D'UNE ASSOCIATION D'ATTRIBU
TS MAKE_RULE_OR_CLASS_NODE ( SLINE( PRIOR_F_COL.PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ); TENTER LA CREATION D'UN NOUVEAU REGLE-(OU LE RAMENE
R S'IL-EXISTE DEJ) GET_TOKEN; ALLER-CHERCHER LE PREMIER-NOM D'ATTRIBUT OU LE ELSE-IF SLINE( F_COL.F_COL + TOKEN_LENGTH - 1
) = ":" THEN LEXEME MARQUANT-UNE DEFINITION DE CLASSE CONTEXT := IN_CLASS; GARDER UNE TRACE DE CE FAIT : DEFINITION D'UNE
CLASSE MAKE_RULE_OR_CLASS_NODE ( SLINE( PRIOR_F_COL.PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ); TENTER LA CREATION D'UN NOUVEAU REGLE-(OU LE RA
MENER S'IL-EXISTE DEJ) GET_TOKEN; ALLER-CHERCHER UN COMPOSANT DE CLASSE WHILE SLINE( F_COL.F_COL + TOKEN_LENGTH - 1 ) = "/"
LOOP JUSQU' LA FIN DE LA DEFINITION DE CLASSE-IF TOKEN_IS_NAME THEN SI L'ON A UN NOM (PAS UNE "-" SEPARATRICE) MAKE_MEMBER ( S
LINE( F_COL.F_COL + TOKEN_LENGTH - 1 ); CREER-UN MEMBRE-DE CLASSE-END IF; GET_TOKEN; AVANCER AU LEXEME SUIVANT END LOOP;
ELSE-IF SLINE( F_COL.F_COL + TOKEN_LENGTH - 1 ) = ";" THEN SEPARATEUR DU TYPAGE GET_TOKEN; AMENER UN SEQ OU LE NOM DU
TYPE IF SLINE( F_COL.F_COL + TOKEN_LENGTH - 1 ) = "seq" THEN C'EST-UN SEQ-GET_TOKEN; PRENDRE LE OF OU LE NOM DE TYPE-IF S
LINE( F_COL.F_COL + TOKEN_LENGTH - 1 ) = "of" THEN-SEQUENCE- ON A LE OF- GET_TOKEN; PRENDRE LE NOM DE TYPE-END IF; MAKE_ATTR( SLINE( PRIOR
F_COL.PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ), -SEQ, -1 ); AJOUTER UN ATTRIBUT SEQUENCE- SLINE( F_COL.F_COL + TOKEN_LENGTH - 1 ) avec SON TY
PAGE-IF ELSE-IF SLINE( PRIOR_F_COL.PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ) = "lx comments" THEN SI CE-N'EST PAS UN ATTRIBUT LX COMMENTS-M
AKE_ATTR( SLINE( PRIOR_F_COL.PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ), NORMAL, -1 ); AJOUTER UN ATTRIBUT SIMPLE (NON-SEQ) SLINE( F_COL.F_COL +
TOKEN_LENGTH - 1 ) avec SON TYPAGE-IF ELSE-IF SLINE( PRIOR_F_COL.PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ) = "normal" THEN LEXEME NON RECONNU, PASSER AU SUIVANT
END IF; END LOOP; USER_ROOT := MAKE ( DN_USER_ROOT ); FABRIQUER LE NOUVEAU RACINE-ARBRE D( XD_SOURCENAME, USER_ROOT, STORE_TEX
T ( NOM_TEXTE ); Y METTRE LE
```

```

    XD_IS_CLASS, NODE ) THEN-----| UN NOEUD CLASSE--DECLARE-- THE_NAME:= CONSTANT STRING:= PRINT_NAME( D ( XD_SYMREP,~NODE ) );--BEGIN-- IF THE_N
AME /= "NON_DIANA"-- AND THEN THE_NAME /= "ALL_SOURCE"-- AND THEN THE_NAME /= "TYPE_SPEC"-- AND THEN THE_NAME /= "STANDARD_IDL"-- THEN--
PUT_LINE ( "**** PARTITION INATTENDUE = " & THE_NAME );-- END IF;-- PUT ( THE_NAME );-- IF NOT IS_CLASS THEN-- PUT ( CFILE, THE_NAME );-- END IF;
--END;-- ELSE-----| UN NOEUD DE REGLE D'ATTRIBUTION--PUT( "...");-- END IF;-- ELSE-----| PAS CLASSE DE BASE-- CLA
SS_PATH(~PARENT, IS_CLASS );-----| REMONTER VERS LA CLASSE DE BASE-- PUT( " > " & PRINT_NAME( D ( XD_SYMREP, NODE ) ) );-- IF NOT IS_CL
ASS THEN--PUT ( CFILE, " > " & PRINT_NAME ( D ( XD_SYMREP, NODE ) ) );-----| REPETER-- END IF;-- END-IF;-- END CLASS_PATH;-- BEGIN-- C
REATE ( NFILE,~OUT_FILE,~NOM_TEXTE-& "NODES.txt");-----| FICHER INFORMATION TEXTE-DES NOEUDS-- CREATE ( CFILE,~OUT_FILE,~NOM_TEXTE-& "_CLASS_
.txt");-----| FICHER INFORMATION TEXTE-HIERARCHIE DES CLASSES-- SET_OUTPUT ( NFILE );-- PUT_LINE ( "----- ARBORESCENCE IDL -----");-- WHILE~N
0130 OT IS_EMPTY ( NODE_LIST ) LOOP-- POP~( NODE_LIST, RULE_NODE );-- DECLARE-- IS_A_CLASS:= CONSTANT BOOLEAN:= DB ( XD_IS_CLASS, RULE_NO
DE );-- BEGIN-- DECLARE--RULE_CLASS_NAME:= CONSTANT STRING:= PRINT_NAME ( D (~XD_SYMREP, RULE_NODE ) );-- BEGIN--IF IS_A_CLASS THEN
-----| DEFINIT UNE CLASSE-- PUT ( "{" & RULE_CLASS_NAME-& "}" );--ELSE-- PUT ( RULE_CLASS_NAME );--END IF;-- END;-- PUT ( ASCII_HT
& "PATH " );-- CLASS_PATH ( RULE_NODE, IS_CLASS=> DB ( XD_IS_CLASS, RULE_NODE ) );-- IF NOT IS_A_CLASS THEN-----| PAS UNE CLASSE--NEW
_LINE ( CFILE );-- END IF;-- ITEM_LIST := LIST ( RULE_NODE );-----| LISTE-DES ATTRIBUTS OU DES MEMBRES-- IF NOT IS_EMPTY ( ITEM
_LIST ) THEN--IF IS_A_CLASS THEN-----| DEFINIT UNE CLASSE-- PUT ( " := " );--END IF;-- END IF;-- NEW_LINE;-- END;-- WHILE N
OT IS_EMPTY ( ITEM_LIST ) LOOP-- POP ( ITEM_LIST, ITEM);-- IF ITEM.TY = DN_ATTR THEN-----| ATTRIBUT--PUT ( ASCII_HT & "> " & PRINT
_NAME ( D( XD_SYMREP, ITEM ) ) & ASCII_HT & ": " );--IF DI ( XD_ATTR_ID,~ITEM ) < 0 THEN-----| ATTRIBUT SEQUENCE-- PUT ( "SEQ OF " );--END IF;--PUT
_LINE ( PRINT_NAME ( D ( XD_ATTR_TYPE, ITEM ) ) );-----| TYPAGE-- ELSE-----| MEMBRE DE CLASSE--PUT ( ASCII_HT & PRINT_NAME (~D ( XD_SYMRE
P, ITEM ) ) );-----| NOM DU MEMBRE--DEFLIST := LIST ( D( XD_SYMREP, ITEM ) );--IF IS_EMPTY ( DEFLIST ) OR ELSE HEAD ( DEFLIST ).TY /= DN_CLASS_NODE
0140 THEN-- PUT ( " ????? " );--END IF;--NEW_LINE;-- END IF;-- END-LOOP;-- NEW_LINE;-- END LOOP;-- SET_OUTPUT ( STANDARD_OUTPUT );-- C
LOSE~( NFILE );-- CLOSE~( CFILE );-- EXCEPTION-- WHEN OTHERS =>-- CLOSE ( NFILE);-- CLOSE ( CFILE);-- PUT_LINE ( "ERREUR A L IMPRES
SION" );-- END PRINT_IDL;--BEGIN-- OPEN ( IFILE, IN_FILE, ".../idl/" &~NOM_TEXTE-& ".idl" );-----| FICHER SOURCE IDL-- PUT_LINE ( "LE FICHER :
" & NOM_TEXTE & ".idl EST OUVERT " );-- CREATE_IDL_TREE_FILE ( NOM_TEXTE & ".lar");-----| FICHER D'ARBRE-IDL-- PUT_LINE ( "LE FICHER : " & NOM
_TEXTE & ".lar EST CREE" );-- SOURCE_LIST := (TREE_NIL,TREE_NIL);-- PUT_LINE ( "PROCESS IDL ..." );-- PROCESS_IDL;-- PUT_LINE ( " OK " );-- LIST ( TREE_R
OOT,~SOURCE_LIST );-- CLOSE (~IFILE );-- CHECK_IDL;-- PRINT_IDL;-- CLOSE_IDL_TREE_FILE;--EXCEPTION-- WHEN NAME_ERROR =>-- PUT_LINE ( "LE FICHER D
ESCRIPTION : " & ".../idl/" &~NOM_TEXTE-& ".idl EST INTROUVABLE" );--END IDL_READ;--
-----|END IDL_READ;--

```